Processes

An operating system executes a variety of programs that run as a process.

Process–

A process is a program in execution. Its instructions run sequentially, and in a single-processor system, a process cannot execute multiple instructions in parallel.

Multiple parts of a Process

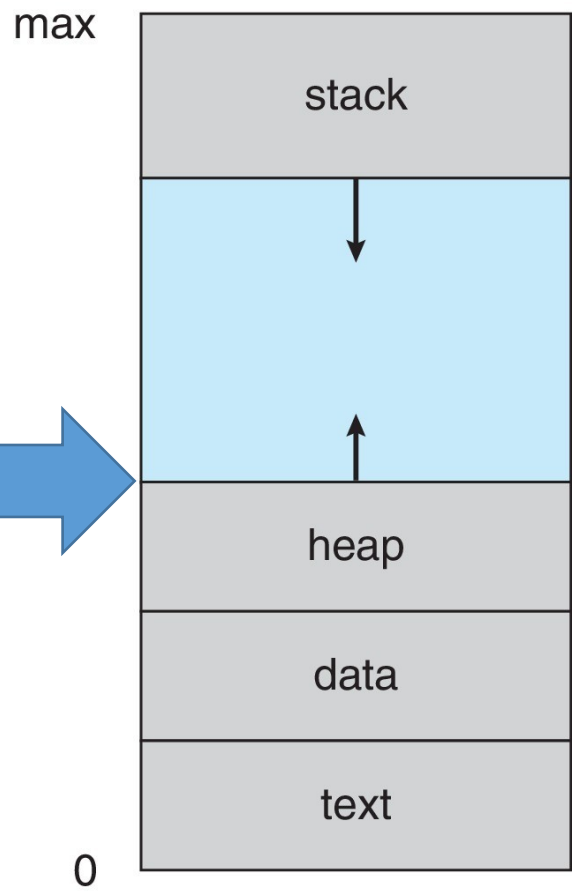
- The program code, also called **text section**
- Current activity including **program counter**, processor registers
- **Stack** containing temporary data
  - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time

Processes

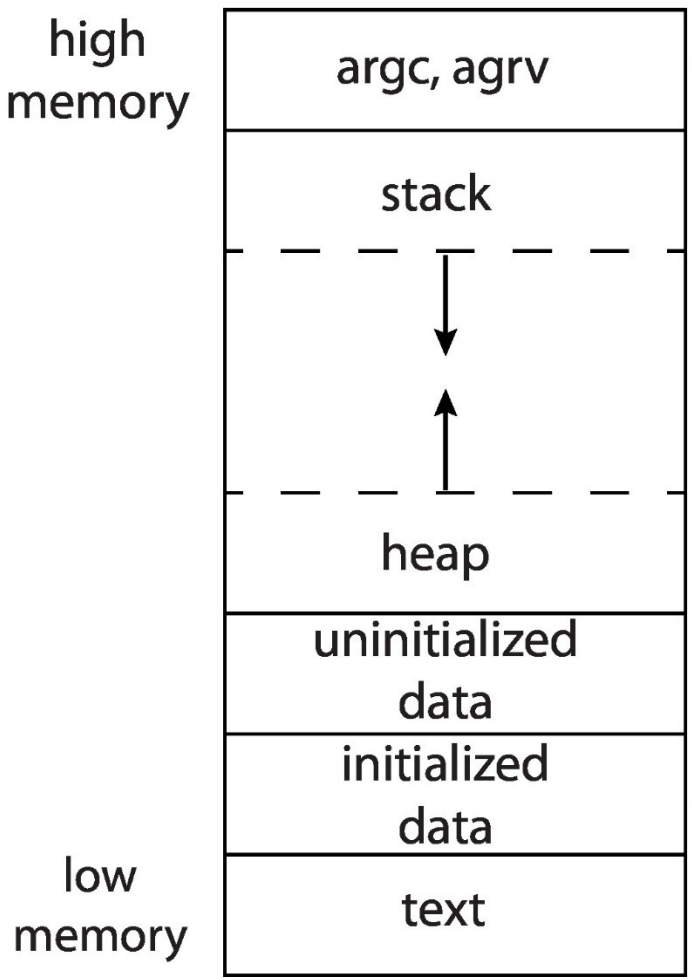
- Program is **passive** entity stored on disk (**executable file**); whereas a process is **active** entity.
- A Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program, or single user can run multiple entities of the same program.

Processes in Memory

This figure shows how the different components of a process are stored in main memory



Memory Layout of a C Program

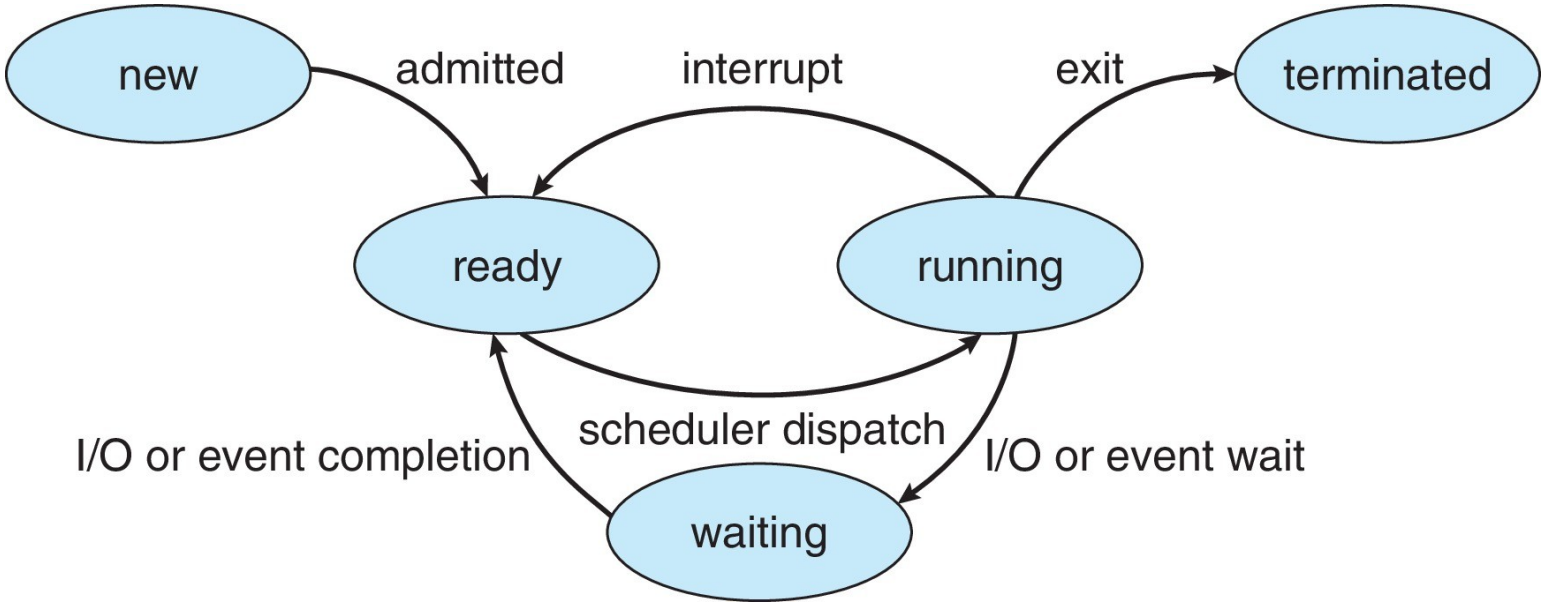



```
#include <stdio.h>
#include <stdlib.h>

int x;
int y = 15;

int main(int argc, char *argv[])
{
    int *values;
    int i;

    values = (int *)malloc(sizeof(int)*5);

    for(i = 0; i < 5; i++)
        values[i] = i;

    return 0;
}
```

Processes States

As a process executes, it changes **state**

- **New**: The process is being created
- **Ready**: The process is waiting to be assigned to a processor
- **Running**: Instructions are being executed
- **Waiting**: The process is waiting for some event to occur
- **Terminated**: The process has finished execution



Process Control Block (PCB)

(also called **task control block**)

A Process Control Block (PCB) is a data structure that stores all the information associated with each process.

It include- Processs ID

Process state – running, waiting, etc.

Program counter – location of instruction to next execute

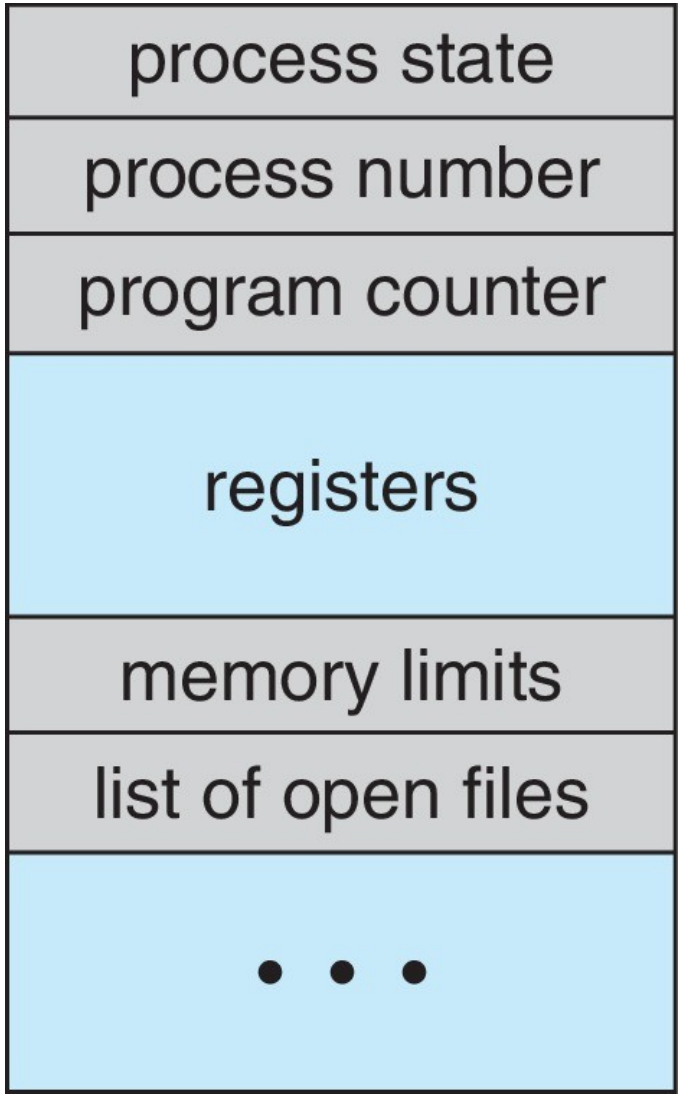
CPU registers – contents of all process-centric registers

CPU scheduling information- priorities, scheduling queue pointers

Memory-management information – memory allocated to the process

Accounting information – CPU used, clock time elapsed since start, time limits

I/O status information – I/O devices allocated to process, list of open files



Process Scheduling

Process scheduler selects among available processes for next execution on CPU core

Goal -- Maximize CPU use, quickly switch processes onto CPU core

Maintains **scheduling queues** of processes

- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Wait queues** – set of processes waiting for an event (i.e., I/O)
- Processes migrate among the various queues

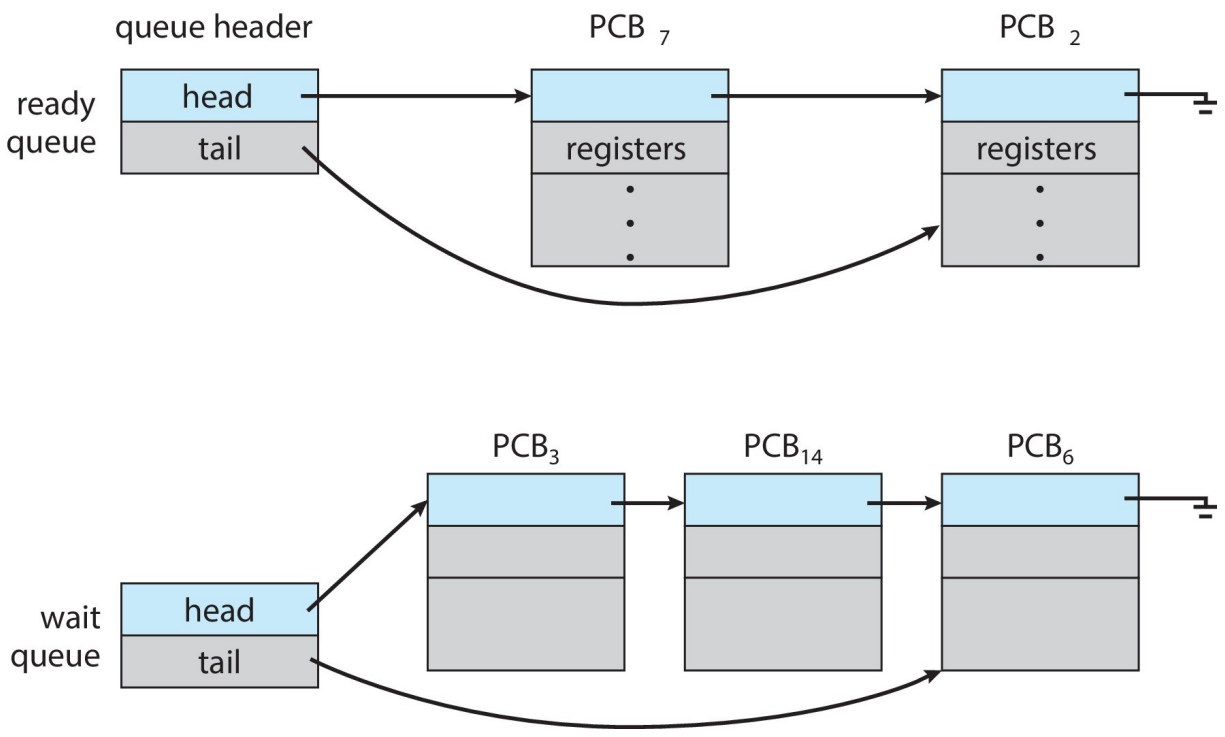
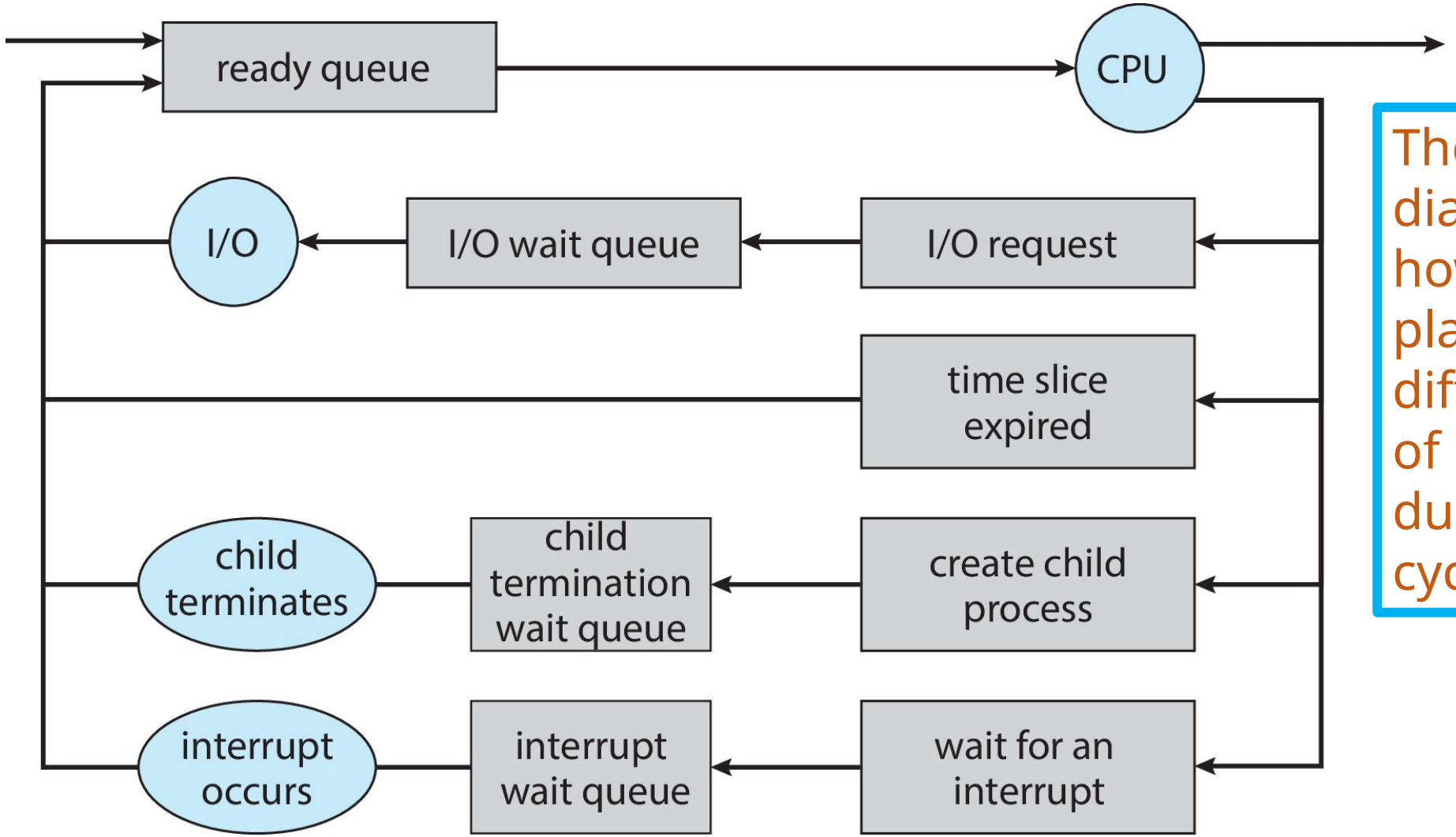


Figure: Ready and Wait queues

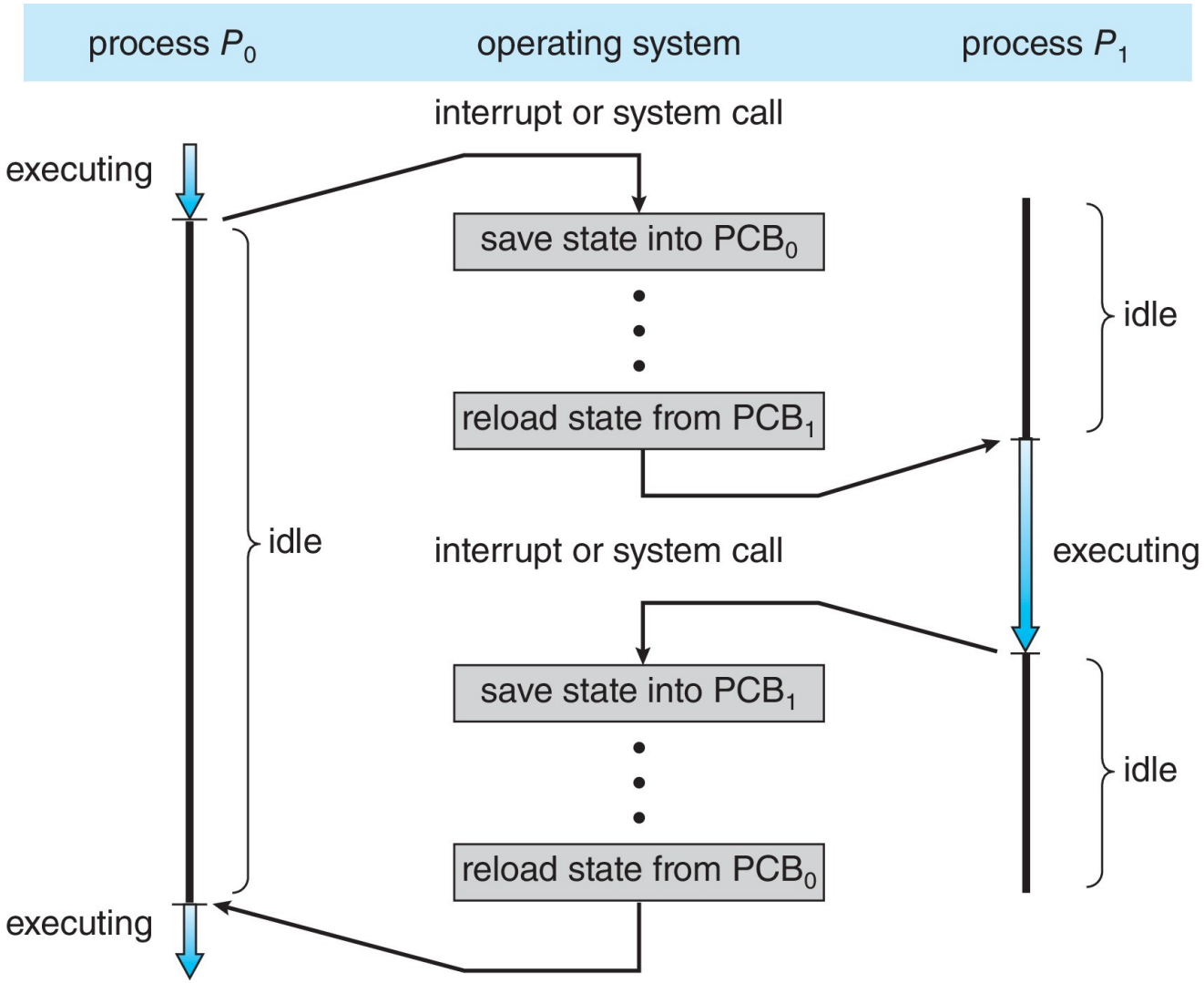
Representation of Process Scheduling: The Queueing Diagram



The queueing diagram shows how a process is placed in different types of queues during its life cycle.

Context Switch

- Context switching in an operating system involves saving the context or state of a running process so that it can be restored later, and then loading the context or state of another process and run it.
- Context-switch time is pure overhead; the system does no useful work while switching
- The more complex the OS and the PCB ➔ the longer the context switch.



Context Switch

Context Switching Triggers

The three different categories of context-switching triggers are as follows.

- Interrupts
- Multitasking
- User/Kernel switch

Interrupts: When a CPU requests that data be read from a disc, if any interruptions occur, context switching automatically switches to a component of the hardware that can handle the interruptions more quickly.

> **Example:** Suppose a user is watching a video on a computer, and suddenly, an external USB device is plugged in. The CPU temporarily switches context to handle the USB detection and then returns to playing the video.

Context Switch

Context Switching Triggers

Multitasking: The ability for a process to be switched from the CPU so that another process can run is known as context switching. When a process is switched, the previous state is retained so that the process can continue running at the same spot in the system.

> **Example:** A user is working on a document while also listening to music. When the user starts running a heavy software application, the operating system switches the CPU from playing music to processing the application, ensuring that both tasks run smoothly without interruption.

Kernel/User Switch: This trigger is used when the OS needed to switch between the user mode and kernel mode.

> **Example:** A program running in user mode requests to read a file from the disk. Since file access requires privileged operations, the CPU switches from user mode to kernel mode, allowing the OS to handle the request safely before returning control to the user program.